

Pipeline CI/CD — FutureKawa

Documentation du pipeline GitLab CI

Projet : FutureKawa

Version : 1.0

Date : 02/07/2026

Équipe : Rudy · Florian · Noé · Josselin

Sommaire

Pipeline CI/CD — FutureKawa

01 — Stage Lint

02 — Stage Test

03 — Stage Build

04 — Stage Staging (Smoke Test)

05 — Stage Sécurité

06 — Stage Deploy

Pipeline CI/CD — FutureKawa

Projet	FutureKawa
Document	Documentation du pipeline CI/CD
Version	1.0
Date	02/07/2026
Équipe	Rudy · Florian · Noé · Josselin

Vue d'ensemble

Le pipeline CI/CD est défini dans `.gitlab-ci.yml` à la racine du dépôt. Il utilise **GitLab CI** avec des runners Docker.

Stages

```
lint → test → build → staging → security → deploy
```

Stage	Rôle	Déclenchement
lint	Vérification syntaxique et style	MR + push sur toute branche
test	Tests unitaires et intégration	MR + push sur toute branche
build	Build et push des images Docker	dev et main uniquement
staging	Smoke test avec les images buildées	dev et main uniquement
security	SAST + détection de secrets	Toutes branches
deploy	Validation Terraform + gate manuel	main uniquement

Navigation

#	Document	Contenu
1	Lint	Vérifications ESLint, Ruff, ShellCheck
2	Tests	Jobs de test par service, couverture, artifacts
3	Build	Construction des images Docker, registry, tags
4	Staging	Smoke test de la stack CI
5	Sécurité	SAST GitLab, détection de secrets
6	Deploy	Gate manuel, Terraform validate

Règles de déclenchement

Les jobs CI sont **conditionnels** : ils ne tournent que si des fichiers du service concerné ont changé. Cela évite de lancer les tests de l'API siège quand seul le frontend exploitation a été modifié.

```
rules:
  - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    changes:
      - API-siege/**/*
  - if: '$CI_PIPELINE_SOURCE == "push"'
    changes:
      - API-siege/**/*
```

Résumé des jobs

Job	Stage	Service	Image
<code>lint_simu_senso</code> <code>r</code>	lint	simu-sensor	node:20

Job	Stage	Service	Image
lint_front_siege	lint	front-siege	node:20
lint_front_exploitation	lint	front-exploitation	node:20
lint_api_siege	lint	API Siège	python:3.12-slim
lint_api_exploitation	lint	API Exploitation	python:3.12-slim
lint_broker	lint	broker	shellcheck-alpine
simu_sensor_test	test	simu-sensor	node:20
api_siege_test	test	API Siège	python:3.11-alpine
api_exploitation_test	test	API Exploitation	python:3.11-alpine
front_siege_test	test	front-siege	node:20
front_exploitation_test	test	front-exploitation	node:20
broker_acl_test	test	broker	docker:24
build_api_siege	build	API Siège	docker:24
build_api_exploitation	build	API Exploitation	docker:24
build_front_siege	build	front-siege	docker:24
build_front_exploitation	build	front-exploitation	docker:24
build_broker	build	broker	docker:24
smoke_test	staging	toute la stack	docker:24
semgrep-sast	security	tout le code	GitLab template
secret_detection	security	tout le code	GitLab template

Job	Stage	Service	Image
deploy_prod	deploy	infra/	hashicorp/ terraform:1.9

01 — Stage Lint

Rôle

Le stage `lint` vérifie la **qualité syntaxique et stylistique** du code avant de lancer les tests. Un job échoué ici bloque la suite du pipeline.

Jobs

`lint_simu_sensor` — ESLint (Node 20)

Vérifie le code TypeScript/Vue du simulateur de capteur.

```
image: node:20
script:
  - cd simu-sensor
  - npm ci
  - npx eslint .
```

Déclenché si : fichiers `simu-sensor/**/*.*` modifiés.

`lint_front_siege` — ESLint (Node 20)

Vérifie le code TypeScript/Vue du frontend siège.

```
image: node:20
script:
  - cd front-siege
  - npm ci
  - npx eslint .
```

Déclenché si : fichiers `front-siege/**/*.*` modifiés.

`lint_front_exploitation` — ESLint (Node 20)

Vérifie le code TypeScript/Vue du frontend exploitation.

```
image: node:20
script:
  - cd front-exploitation
  - npm ci
  - npx eslint .
```

Déclenché si : fichiers `front-exploitation/**/*.*` modifiés.

`lint_api_siege` — **Ruff (Python 3.12)**

Vérifie le code Python de l'API siège avec Ruff (linter + formateur Python ultra-rapide).

```
image: python:3.12-slim
script:
  - pip install -q ruff
  - cd API-siege
  - ruff check .
```

Déclenché si : fichiers `API-siege/**/*.*` modifiés.

`lint_api_exploitation` — **Ruff (Python 3.12)**

Même chose pour l'API exploitation.

```
image: python:3.12-slim
script:
  - pip install -q ruff
  - cd API-exploitation
  - ruff check .
```

Déclenché si : fichiers `API-exploitation/**/*.*` modifiés.

`lint_broker` — **ShellCheck**

Vérifie les scripts shell du broker MQTT (`entrypoint.sh` , `tests/test_acl.sh`) avec ShellCheck, le linter shell de référence.


```
image: koalaman/shellcheck-alpine:stable
script:
  - shellcheck broker/entrypoint.sh broker/tests/test_acl.sh
```

Déclenché si : fichiers `broker/**/*` modifiés.

En cas d'échec

Outil	Commande locale équivalente
ESLint	<code>cd <service> && npx eslint .</code>
Ruff	<code>cd API-<service> && ruff check .</code> <code>--fix</code>
ShellCheck	<code>shellcheck broker/entrypoint.sh</code>

02 — Stage Test

Rôle

Le stage `test` exécute tous les tests automatisés en CI. Chaque job tourne dans un container isolé, produit un rapport JUnit et un rapport de couverture au format Cobertura.

Jobs

`api_siege_test` — Django (Python 3.11)

Lance les tests unitaires et d'intégration de l'API siège avec couverture.

```
cd API-siege
pip install -r requirements.txt unittest-xml-reporting coverage
coverage run --source=siege manage.py test siege.tests
coverage report
coverage xml -o coverage.xml
```

Variables : `DJANGO_SETTINGS_MODULE: core.settings.test`

Artifacts : - Rapport JUnit : `API-siege/test-reports/*.xml` - Couverture Cobertura : `API-siege/coverage.xml`

`api_exploitation_test` — Django (Python 3.11)

Identique pour l'API exploitation.

```
cd API-exploitation
coverage run --source=exploitation manage.py test exploitation.tests
```

Artifacts : - Rapport JUnit : `API-exploitation/test-reports/*.xml` - Couverture Cobertura : `API-exploitation/coverage.xml`

`simu_sensor_test` — Vitest (Node 20)

Lance les tests Vitest du simulateur de capteur avec couverture V8.

```
cd simu-sensor
npm ci
npm run test:coverage
```

Artifacts : - Rapport JUnit : `simu-sensor/junit.xml` - Couverture
Cobertura : `simu-sensor/coverage/cobertura-coverage.xml`

`front_siege_test` — Vitest (Node 20)

Lance les tests Vitest du frontend siège.

```
cd front-siege
npm ci
npm run test:coverage
```

Artifacts : - Rapport JUnit : `front-siege/junit.xml` - Couverture
Cobertura : `front-siege/coverage/cobertura-coverage.xml`

`front_exploitation_test` — Vitest (Node 20)

Lance les tests Vitest du frontend exploitation.

```
cd front-exploitation
npm ci
npm run test:coverage
```

`broker_acl_test` — Docker-in-Docker

Test spécial : lance un container Mosquitto et vérifie les règles ACL avec le script `broker/tests/test_acl.sh`.

```
docker network create mqtt-test-net
docker build -t broker-test broker/
docker run -d --name broker-test --network mqtt-test-net broker-test
```

```
docker run --rm eclipse-mosquitto:2.0 sh /tests/test_acl.sh broker-  
test 1883
```

Vérifie que les publications/souscriptions MQTT autorisées fonctionnent et que celles interdites sont bien rejetées.

Résumé des résultats

Service	Tests	Couverture
API Siège	~125	96 %
API Exploitation	~150	96 %
Front Siège	~250	~95 %
Front Exploitation	~165	~97 %
Simu-Sensor	~166	~95 %
Broker MQTT	8 assertions ACL	100 %

Tests non inclus en CI

Les tests E2E Playwright ne tournent pas en CI car ils nécessitent une stack complète. Ils sont disponibles en local :

```
make test-simu-sensor-e2e
```

03 — Stage Build

Rôle

Le stage `build` construit les images Docker de chaque service et les pousse dans le **registry GitLab**. Il ne se déclenche que sur les branches `dev` et `main`.

Template commun

Tous les jobs de build partagent un template YAML (`&build_template`) :

```
stage: build
image: docker:24
services:
  - docker:24-dind
before_script:
  - docker login -u "$CI_REGISTRY_USER" -p "$CI_REGISTRY_PASSWORD"
"$CI_REGISTRY"
  - |
    if [ "$CI_COMMIT_BRANCH" = "main" ]; then
      export TAG_MOVING="latest"
      export TAG_SHA="main-$CI_COMMIT_SHORT_SHA"
    else
      export TAG_MOVING="dev-latest"
      export TAG_SHA="dev-$CI_COMMIT_SHORT_SHA"
    fi
rules:
  - if: '$CI_COMMIT_BRANCH == "dev"'
  - if: '$CI_COMMIT_BRANCH == "main"'
```

Jobs

build_api_siege

```
docker build -t "$REGISTRY/futurekawa-api-siege:$TAG_SHA" API-siege/
docker tag ... :$TAG_MOVING
docker push ... :$TAG_SHA
docker push ... :$TAG_MOVING
```

build_api_exploitation

Idem pour `API-exploitation/` .

build_front_siege

Idem pour `front-siege/` .

build_front_exploitation

Idem pour `front-exploitation/` .

build_broker

Idem pour `broker/` .

Stratégie de tags

Branche	Tag mobile	Tag SHA (immuable)
dev	dev-latest	dev-<sha court>
main	latest	main-<sha court>

Le tag SHA est utilisé par le job `smoke_test` pour tester exactement les images qui viennent d'être buildées.

Images buildées

Image	Registry
futurekawa-api-siege	registry.gitlab.com/<groupe>/futurekawa/
futurekawa-api-exploitation	idem

Image	Registry
futurekawa-front-siege	idem
futurekawa-front-exploitation	idem
futurekawa-broker	idem

04 — Stage Staging (Smoke Test)

Rôle

Le stage `staging` lève la stack backend complète à partir des images **tout juste buildées** et vérifie que les deux APIs répondent sur `/api/health/`. Si ce test échoue, le pipeline s'arrête avant le deploy.

Job : `smoke_test`

Principe

Le smoke test utilise `docker-compose.ci.yml`, une version allégée de la stack (pas de frontends, pas de simu-sensor). Les images viennent du registry GitLab, pas d'un bind-mount du code source.

```
stage: staging
image: docker:24
services:
  - docker:24-dind
```

Script

```
# 1. Récupère les images buildées à ce SHA
docker compose -f docker-compose.ci.yml pull

# 2. Lève la stack
docker compose -f docker-compose.ci.yml up -d

# 3. Attend que l'API siège réponde (30 tentatives × 2 s)
until docker compose exec -T api-siege wget -q --spider -T 3 http://
127.0.0.1:5000/api/health/
do sleep 2; done

# 4. Assert API siège
docker compose exec -T api-siege wget -q --spider http://
```



```
127.0.0.1:5000/api/health/
```

```
# 5. Même séquence pour api-exploitation
```

```
# ...
```

```
# 6. État final
```

```
docker compose -f docker-compose.ci.yml ps
```

Nettoyage (after_script)

```
docker compose -f docker-compose.ci.yml logs --tail=100
```

```
docker compose -f docker-compose.ci.yml down -v
```

Pourquoi `127.0.0.1` et pas `localhost` ?

Dans ces containers CI, `localhost` résout en IPv6 (`::1`) en premier, mais les serveurs Django n'écoutent qu'en IPv4. Utiliser `127.0.0.1` force l'IPv4 et évite un faux négatif de healthcheck.

Déclenchement

Uniquement sur les branches `dev` et `main` , après le stage `build` .

05 — Stage Sécurité

Rôle

Le stage `security` exécute des analyses de sécurité statiques sur le code. Il utilise les **templates GitLab CI gratuits** disponibles sur le tier Free.

Templates inclus

```
include:
  - template: Jobs/SAST.gitlab-ci.yml
  - template: Jobs/Secret-Detection.gitlab-ci.yml
```

Job : `semgrep-sast` — Analyse statique de sécurité

SAST (Static Application Security Testing) analyse le code source à la recherche de vulnérabilités connues (injections, mauvaise gestion des secrets, failles d'authentification...).

GitLab utilise **Semgrep** pour les projets Python et JavaScript/TypeScript.

Résultats : disponibles dans l'onglet « Sécurité » de la MR GitLab et dans les artifacts du job.

Job : `secret_detection` — Détection de secrets

Analyse tous les commits de la branche pour détecter des secrets commités par erreur : - Clés API - Mots de passe - Tokens JWT - Clés SSH / certificats

Important : ce job analyse l'**historique Git**, pas seulement les nouveaux fichiers. Un secret supprimé dans un commit ultérieur sera quand même détecté.

Positionnement dans le pipeline

Les jobs SAST et Secret Detection sont assignés au stage `security` dans `.gitlab-ci.yml` pour les regrouper visuellement. Par défaut, les templates GitLab les mettent dans `test` — le fichier CI les réassigne explicitement :

```
semgrep-sast:
  stage: security

secret_detection:
  stage: security
```

Ce qui n'est pas inclus (réservé GitLab Ultimate)

Feature	Disponibilité
Dependency Scanning	Ultimate uniquement
Container Scanning	Ultimate uniquement
DAST	Ultimate uniquement

06 — Stage Deploy

Rôle

Le stage `deploy` est le dernier de la pipeline. Il valide la configuration Terraform et attend une **action manuelle** avant de s'exécuter. Il ne déploie rien automatiquement.

Job : `deploy_prod`

```
stage: deploy
image:
  name: hashicorp/terraform:1.9
  entrypoint: [""]
when: manual
rules:
  - if: '$CI_COMMIT_BRANCH == "main"'
```

Script

```
# Validation Terraform – Application Siège
cd infra/siege
terraform init -backend=false -input=false
terraform validate

# Validation Terraform – Application Exploitation
cd ../exploitation
terraform init -backend=false -input=false
terraform validate

echo "Terraform valide. plan/apply volontairement absents."
```

Ce que ce job fait (et ne fait pas)

Action	Status
Valide la syntaxe Terraform	Oui
Vérifie la cohérence des modules	Oui
Exécute <code>terraform plan</code>	Non (nécessite un serveur cible)
Exécute <code>terraform apply</code>	Non (nécessite un serveur cible)
Déploie les containers	Non

Pourquoi `when: manual` ?

Le déploiement en production est une action délibérée qui ne doit jamais arriver automatiquement. L'opérateur doit cliquer sur le bouton dans l'interface GitLab CI pour déclencher ce job.

Vers un déploiement réel

Pour activer un vrai déploiement, il faudrait :

1. Un VPS cible avec Docker installé et accessible depuis le runner CI
2. Les variables Terraform sensibles configurées dans GitLab CI/CD
Settings → Variables :
3. `TF_VAR_db_password`
4. `TF_VAR_secret_key`
5. `TF_VAR_docker_host` (IP ou socket SSH du VPS)
6. Décommenter les étapes `terraform plan` et `terraform apply` dans le script